

S3GBR
EPITA 2025-2026

Fasquel Wachirawit
Auguste Noé
Lamarre Timothy
Camarasat-Birabent Hugo

OCR PROJECT REPORT

Third semester Project

Summary:

Page 3-7 : Role and Mission

Page 8-12 : Challenges, Technological, Algorithmic Choices, and Justifications

page 12-14 : Justification of Design choices

Page 14-15 : Current Progress and Next Steps

Page 15 : Bibliography

A) Role and Mission :

I) Pretreatment :

My name is Wachirawit Noel Fasquel and my main responsibility in the project was image preprocessing, which included loading the image, converting it to grayscale, binarizing it using the Otsu thresholding method, and implementing a manual rotation function. This step ensures that the image is correctly formatted for the subsequent grid and character detection stages handled by other team members.

My main objectives for the first defence :

1. Image loading and color removal, and
2. Manual rotation of the image.

I approached these objectives in a logical and sequential manner. The first step was to correctly load the image into memory. For this, I implemented a struct capable of storing all essential data: the image's width, height, number of channels, and pixel array.

Once the image could be successfully loaded, I moved on to color removal by converting it to grayscale. This was achieved by calculating a weighted sum of the red, green, and blue components of each pixel. This transformation reduced the image to a single channel, simplifying the later processing stages.

The next step was binarization using Otsu's thresholding method. This algorithm calculates the histogram of the grayscale image, identifies the threshold that minimizes the intra-class variance (i.e., the variance within black and white pixel groups), and then assigns each pixel a value of either black or white based on that threshold. In the program, this corresponds to replacing each pixel value with 1 for black and 0 for white, resulting in a binary image.

After completing the binarization, I implemented the rotation system, allowing the user to manually rotate the image by 90°, 180°, or 270°. This feature is essential for handling input files that may not be properly oriented, ensuring that the grid detection algorithm receives a correctly aligned image.

At the current stage, the manual rotation and binarization parts are fully functional. The next steps for the final defense include implementing automatic rotation, as well as additional preprocessing features such as contrast enhancement and noise reduction. Given the current rate of progress, these remaining objectives are expected to be completed by the time of the final presentation.

II) Localization and Extraction :

My name is Noé Auguste, and my primary responsibility in the project was the localization and extraction of the word search grid and the word list from the preprocessed image. This involved developing algorithms to identify the precise coordinates of the grid and the list areas, segment the grid into individual cells, and isolate the words in the list for later character recognition by Timothy, another team member. This step was critical, as the accuracy of the entire solving process depended on the correct isolation of these elements.

For the first defense, my main objectives were :

1. Locating the grid and the word list on the image.
2. Isolating and extracting each individual cell of the grid.
3. Implementing a visualization system to debug the localization process.

I approached the localization problem by first analyzing the structural properties of a binarized image. The grid, being a large, regular structure of lines and boxes, presented a distinct signature that could be identified by analyzing pixel projections along the X and Y axes. By summing the black pixels per row and per column, I could identify the dense bands of lines that defined the grid's boundaries.

Once the grid's region was identified and isolated, the same principle was applied on a smaller scale to segment it into individual cells. This allowed the program to extract a small image for each letter, which would be passed to the neural network.

Simultaneously, I worked on locating the word list. This was often positioned outside the grid and could be identified as a contiguous block of text. The challenge was to reliably separate this block from potential noise and then to segment it into individual words and subsequently into individual letters, similar to the grid cell extraction.

I am fully aware that this method, while highly effective for the clean, well-defined structure of Level 1 images, has its limitations. It primarily relies on the presence of clear borders and a clear separation between the grid and other elements. For more complex images, such as those in Level 3 where the grid may lack borders or where significant extraneous text is present, this approach would likely fail. Therefore, a key objective for the final defense is to enhance the grid and word list recognition algorithms to handle these more complex cases by incorporating more advanced contour analysis and pattern recognition techniques. The current implementation

serves as a robust and verifiable foundation upon which to build this more advanced functionality.

III) Neural Network :

My name is Timothy Lamarre, and for this project I was mainly responsible for the Neural Network and the character recognition. I first implemented a proof of concept (POC) to verify the correct behavior of a basic neural network. The objective was to design, train, and validate a small feedforward network capable of learning a simple logical function, the XNOR function ($A \cdot B + \neg A \cdot \neg B$). This task serves as an introductory test to ensure that the network's mathematical operations (forward propagation, activation, backpropagation, and weight updates) are all implemented correctly before scaling up to more complex image recognition.

For the first defense, my main objectives were :

1. Implement a basic neural network to understand and validate the fundamental learning mechanisms.
2. Design and train a minimal architecture (2-2-1 structure) to learn the XNOR logical function.
3. Validate the correctness of the implementation by achieving successful learning of the XNOR truth table.

This network consists of:

- 2 input neurons, corresponding to the Boolean variables A and B,
- 1 hidden layer of 2 neurons, using the sigmoid activation function,
- 1 output neuron, also using a sigmoid activation to produce a binary output between 0 and 1.

After several thousand training epochs, the neural network successfully converged to stable predictions with minimal loss, accurately reproducing the XNOR truth table. During testing, different learning rates were evaluated in order to study their impact on convergence. Lower values, typically between 0.1 and 0.3, resulted in smooth and stable learning, whereas higher rates caused oscillations and instability due to the nonlinear behavior of the sigmoid activation function.

This first proof of concept confirmed that the neural network implementation is both functional and mathematically consistent. It demonstrated the model's ability to learn nonlinear relationships, a fundamental capability required for more complex recognition tasks. These encouraging results provide a solid foundation for the next phase of the project, which will involve extending the architecture to handle multi-class classification using a softmax output layer and cross-entropy loss, enabling the recognition of letters from binarized images as part of the OCR system.

IV) Solver (Hugo) :

My name is Hugo Camarasa Birabent and I was in charge of the solver program. The solver program consists in retrieving a the word grid from a given file, looking for a given word in said grid and then printing in the terminal the starting and final coordinates of the found word, however if the word is not found, the program displays "Not found".

For the first defense, my main objectives were :

1. Designing my solving algorithm and imagining all the needed auxiliary functions.
2. Coding the solving algorithm and adapting it to the context
3. Resolving any problem that could happen to deliver a functional program

First of all, the design of the solving algorithm was pretty simple, I just inspired myself from the way I solve this type of problem. During this time, I figured we would need a couple of auxiliary functions such as the retrieval of the grid from the file, the direction in which we would be looking for a word and the search through the grid of words.

Once this was finished, I coded the algorithm and different auxiliary functions in C. At first it was a relatively task simple because I was not really paying attention to the code I was writing, the structure mattered more as I wanted it to be as efficient and understandable as possible. This is why I created other auxiliary functions, to format the word or count the lines of the file for example.

However, when it came to compiling, my code had a lot of errors and that is what I spent most of my time on. As I had done a lot of functions, I had to debug every single one of them until I talked about my problem to a friend and he made me realise that so many functions were not necessary and it helped me a lot.

B) Challenges, Technological, Algorithmic Choices, and Justifications :

I) Pretreatment :

The preprocessing phase presented several technical and algorithmic challenges that required careful planning and experimentation. Each stage of the process—image loading, grayscale conversion, binarization, and rotation—was associated with distinct constraints that influenced the final implementation choices.

➤ Image Loading

The first challenge was the loading of images in various formats while ensuring compatibility across systems. Since the project was developed in C, a low-level language with limited native image-handling capabilities, it was necessary to integrate an external library. The team selected the stb_image.h library, an open-source, single-header C library designed for simplicity and efficiency.

The decision was motivated by several factors:

- It allows direct loading of multiple image formats (JPG, PNG, BMP, and others) without external dependencies.

- Its integration process is minimal, requiring only the definition `#define STB_IMAGE_IMPLEMENTATION` before inclusion.

- It provides raw pixel data, which can be easily manipulated at the byte level, a key requirement for preprocessing operations such as grayscale and binarization.

The main difficulty during this stage was managing the image's memory representation, particularly when dealing with different channel configurations (grayscale, RGB, or RGBA). To maintain consistency, all images were loaded with their original channel count, and conversion operations were adapted accordingly.

➤ Grayscale Conversion

The grayscale transformation involved converting each pixel's RGB triplet into a single intensity value. The adopted formula—

$$Gray = 0.2126 \times R + 0.7152 \times G + 0.0722 \times B$$

—corresponds to the luminance-based model, which reflects human visual perception more accurately than a simple average of RGB components. This choice ensured a balanced representation of brightness and improved the performance of the subsequent binarization step.

The implementation required iterating over all pixels, extracting their red, green, and blue values, and computing the corresponding grayscale value. Despite its computational cost, this approach offered predictable behavior and reliable visual results, even for high-resolution images.

➤ Binarization and Threshold Selection

The Otsu thresholding algorithm was selected to convert the grayscale image into a binary one. The primary motivation behind this choice was its adaptive nature, as it automatically determines the optimal threshold by analyzing the histogram of the image. This eliminates the need for manual calibration or predefined threshold values, which can vary significantly between images.

The implementation process involved:

1. Calculating the histogram of grayscale intensities (0–255).
2. Computing the intra-class variance for each possible threshold.
3. Selecting the threshold that maximizes the inter-class variance, thus ensuring the most distinct separation between black and white regions.

The main difficulty encountered here was ensuring numerical stability and preventing overflow errors during variance computations, which was resolved by using long and double data types for intermediate calculations.

Finally, the binary output was represented in two forms:

A logical version (values 0 or 1) for computational use.

A visual version (values 0 or 255) for display and debugging.

➤ Manual Rotation

Implementing manual rotation required a careful understanding of pixel mapping within a one-dimensional memory buffer. The rotation of an image by 90°, 180°, or 270° degrees involves recalculating the position of each pixel according to its coordinates in the source matrix.

For instance:

A 90° rotation maps pixel (x, y) to $(y, width - 1 - x)$.

A 180° rotation inverses both axes.

A 270° rotation maps (x, y) to $(height - 1 - y, x)$.

The principal challenge was maintaining memory integrity during rotation, as it requires creating a new buffer for the transformed image before freeing the original one. Furthermore, the visual verification of results highlighted the importance of format consistency, as certain image formats (such as WebP) produced unintended results during early tests.

II) Localization and Extraction :

The main challenge was the inherent difficulty of debugging a computer vision algorithm in a pure C and Vim environment, without the immediate visual feedback of a modern IDE. To overcome this, one of my most crucial algorithmic choices was to implement a visualization function. This system allowed me to generate a new image file where the identified grid boundaries, cells, and word list areas were highlighted. Seeing the program's interpretation directly on the image was the breakthrough that turned a frustrating process of reading core dumps and log outputs into a more manageable, visual debugging task. This was immensely helpful not only for directly revealing off-by-one errors in coordinate calculations and flaws in the segmentation logic, but also for visually fine-tuning and adjusting the various thresholds used in the projection histograms to optimally separate the grid from the background.

Another significant challenge was handling the variance in grid presentations, especially preparing for the future handling of Level 3 images where borders might be absent. My initial implementation relied on finding the strong lines of the border. The choice to use projection histograms was justified by its robustness to slight rotations and its computational simplicity, which allowed for real-time analysis even on higher-resolution images.

III) Neural Network :

The first algorithmic choice was to use a feedforward fully-connected architecture composed of:

- an input layer (2 neurons) representing Boolean inputs A and B,
- a hidden layer (2 neurons) using a sigmoid activation function,

- and a single output neuron with the same activation type.

The sigmoid function was chosen because it introduces non-linearity and outputs continuous values between 0 and 1, which are well-suited for binary logic tasks.

For the weight initialization, I used a Xavier-like method, ensuring that activations and gradients remain in a stable range during training, avoiding early saturation or vanishing gradients.

To evaluate the model's learning, the Mean Squared Error (MSE) was used as the cost function, providing a simple yet effective measure of how far the predictions are from the expected outputs.

IV) Solver (Hugo)

The main challenge encountered during the development of the solver was the debugging part. This was due to the fact that the initial chosen structure was too complex to function correctly.

The searching and solving algorithm used is one of our creation.

In the first version of the solving program, there were nearly 6 auxiliary functions because almost everything in the program was broken down to simpler functions in order to not have an immense primary function. The issue with this way of development is that it was hard to know at every given time what variable was used and how and this led to a lot of Segmentation errors and Memory leaks.

The solution to this was simply to reduce to the maximum the auxiliary functions and insert them into the primary function. This allowed certain problems to be fixed but there was still a problem regarding the grid retrieval because of a misunderstanding of the use of the function "getlines()".

In the end, the issues were fixed and the program finally works as expected.

C) Justification of Design Choices :

I) Pretreatment :

The choices made throughout the preprocessing phase were driven by three main criteria:

Simplicity and transparency : All algorithms were implemented manually, without reliance on high-level image processing libraries, to ensure full understanding and control of the data flow.

Performance and memory safety : Operations were designed to minimize unnecessary allocations and optimize pixel-level manipulation.

Compatibility with later stages : The preprocessing output needed to conform to the expected input structure of the grid and character detection algorithms developed by other team members.

II) Localization and Extraction :

The design choices for the localization module were driven by a pragmatic approach focused on creating a verifiable and debuggable foundation, with a clear path for future enhancement. Each choice was made with the following core principles in mind: transparency of the algorithm's operation, performance within the C environment, and laying the groundwork for increased robustness.

➤ Choice of Projection Histograms for Initial Localization

The primary algorithmic choice was to use projection histograms (summing pixel values per row and column) to locate the grid and word list. This method was selected for several key reasons:

Simplicity and Transparency: Unlike more complex contour detection methods that can be difficult to debug in a low-level language, the histogram approach is conceptually straightforward. The state of the algorithm can be easily logged and, more importantly, visualized, allowing for clear reasoning about its successes and failures.

For Level 1 images, which feature strong, clear borders, this method is computationally efficient and highly reliable. It directly exploits the very structural regularity that defines the target we are searching for, making it the most direct solution for the initial project milestone.

Foundation for Complexity: While acknowledged as insufficient for border-less grids, this method provides a clean output—the bounding box of the grid. This precise coordinate data is a necessary first step upon which more advanced algorithms

(e.g., for detecting alignments of letters without borders) can be built in the next phase.

➤ The Critical Investment in a Visualization Debugging Tool

The decision to dedicate time to implementing a visualization function, despite the project's time constraints, was perhaps the most critical choice for the success of the localization module.

Mitigating Toolchain Limitations: Working in C with Vim, without the live inspection tools of a modern IDE, created a significant debugger deficit. The visualization tool directly compensated for this by providing immediate, intuitive feedback. It transformed abstract numerical outputs (e.g., histogram thresholds, coordinate pairs) into a concrete visual representation.

Enabling Empirical Tuning: This tool was indispensable for adjusting the sensitivity thresholds for the histograms. Instead of guessing values and recompiling, we could visually correlate a threshold change with its direct effect on the detected boundaries, turning a blind process into an empirical one.

Ensuring Team Integration: The visual output served as a universal language when merging the module with the preprocessing stage. It allowed for quick verification that the handoff between the cleaned image and the localization logic was correct, preventing lengthy debugging sessions between team members.

➤ Adherence to Core C Principles

The implementation strictly adhered to C programming best practices to ensure stability and performance.

Memory Safety: Given that the localization process involves creating new image buffers for the segmented cells and visualization outputs, careful manual memory management was paramount. The design ensured that for every malloc there was a corresponding free, preventing memory leaks that could crash the long-running application.

Structured Data for Clarity: The use of structs to represent bounding boxes (with x, y, width, height) and other geometric properties made the code more readable and maintainable, reducing the risk of errors when passing coordinate data between functions.

In summary, the design choices were not aimed at creating a final, all-encompassing solution from the outset. Instead, they were chosen to build a transparent, debuggable, and robust system for the known challenges of Level 1 images, while consciously designing the architecture to be extended and refined for the more complex problems posed by Levels 2 and 3.

III) Neural Network :

The use of a sigmoid activation was motivated by its simplicity and differentiability, which are essential for backpropagation.

Even though more modern activations like ReLU or tanh are often preferred in

large-scale models, the sigmoid remains a good starting point for understanding the mathematical behavior of neurons.

The Xavier initialization was chosen to maintain a consistent variance of activations between layers, ensuring that the learning process starts efficiently without vanishing or exploding gradients.

Finally, the MSE loss function was selected for its simplicity and compatibility with sigmoid activations, providing a straightforward way to observe convergence and measure error reduction across epochs.

IV) Solver :

As explained in the previous part, the use of a single big primary function rather than a smaller one with a lot of auxiliary functions was due to the fact that the data and the variables were less manageable that way.

The algorithm has some optimisations included in order to reduce memory usage and computation time.

D) Current Progress and Next Steps :

I) Pretreatment :

At this current stage of development, all the features that are required for the first defense are ready and fully operational, those includes : image loading using the `std_library`, the change of color from and rgb to grayscale, the application of Otsu's threshold so as to make a clear a binary representation, manual rotation of images is also fully functional and supports 90°, 180° and 270°. Concerning the next steps that are expected to be done by the last defense, we have :

the automatic rotation, that implies creating an algorithm that would take an image and find the right degree to turn the image.

the contrast enhancement, I will be tasked with increasing the visibility of grid lines and handwritten or printed elements.

Noise reduction: In this part, I will have to use filtering method on images so as to remove small irregularities so as to prevent grid detection problems

II) Localization and Extraction :

At this stage, the localization of the grid and word list in well-defined (Level 1) images is fully functional, and the visualization tool is an integral part of my

development and testing process. The process of merging my working localization code with Wachirawit's preprocessing pipeline was a key milestone, and it was highly rewarding to see our two independent modules work together seamlessly.

For the final defense, the next steps are to enhance the robustness of the algorithms to handle Level 2 and Level 3 images, which include dealing with skew, noise, and the absence of clear borders, building upon the solid and debuggable foundation now in place.

III) Neural Network :

At this stage, the neural network successfully learns the XNOR function after several thousand training epochs, reaching stable predictions with very low loss.

Different learning rates were tested: smaller values (around 0.1–0.3) led to smooth and stable convergence, while higher rates produced oscillations due to the sigmoid's nonlinearity.

These results validate the correctness of the implementation and confirm that the network behaves consistently with theoretical expectations.

The next step will involve scaling the network architecture to handle multi-class classification, which is required for the OCR application.

This will include:

- expanding the input layer to handle 28×28 pixel images (784 inputs),
- using a softmax output layer with 26 neurons (one per alphabet letter),
- replacing the MSE loss with cross-entropy,
- training the model on real binarized character images.

This progression from a simple logical function to an image-based recognition task will establish the foundation for the full OCR system, demonstrating both the scalability and flexibility of the neural network implementation.

IV) Solver (Hugo) :

Currently, the solver algorithm is finished and its respective program is functional. The next step would be the display of the resolved grid, the saving of the result and to design and develop the graphical interface for the user to use the program.

E) Bibliography :

stb single-file public domain libraries.

Available at: <https://github.com/nothings/stb>

The stb_image.h and stb_image_write.h libraries are lightweight and public-domain C headers used for loading and saving images in various formats. They were used in this project to handle image input and output during preprocessing.

“A Threshold Selection Method from Gray-Level Histograms”

N. Otsu (1979). *A Threshold Selection Method from Gray-Level Histograms*. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62-66.

https://en.wikipedia.org/wiki/Otsu%27s_method

Bouchet, D. (2024). OCR Word Search Solver Specifications. EPITA.

This project specification document provided the foundational requirements and constraints for the grid and word list localization tasks, defining the three levels of image complexity and the expected outputs.

Bradski, G., & Kaehler, A. (2008). Learning OpenCV. O'Reilly Media.

Although the project was implemented in pure C, the computer vision concepts for object detection and image segmentation discussed in this reference, such as histogram analysis and contour detection, provided a theoretical basis for the development of the localization algorithms.